

PROJECT 3

SQL Data Analysis

DecodeLabs | Data Analytics Internship | Batch 2026

Analyst: Adesola Sobambo

Dataset: Sales Orders | 1,200 Rows | 2023–2025

Tool: DB Browser for SQLite

Date: June 2026

Table of Contents

Section	Title	Page
Introduction	SQL Execution Order & Project Scope	3
Concepts Overview	SQL Concepts Demonstrated Across 16 Queries	4
Q1	Overall Business Summary	5
Q2	Orders by Status	6
Q3	Revenue by Referral Source — All Orders	8
Q4	Top Products by Revenue	10
Q5	Referral Source — Successful Orders Only	12
Q6	Payment Method Breakdown	14
Q7	Coupon Usage & Realisation Rate	16
Q8	Year-on-Year Realisation Rate	19
Q9	High Value Orders — Realised vs Unfiltered (>\$3,000)	22
Q10	High Value Conversion Rate by Channel	27
Q11	Products with 90+ Successful Orders — HAVING	29
Q12	Cancellation & Return Rate by Product	31
Q13	Top 10 Customers — Value & Product Mix	33
Q14	Product Revenue Contribution % — Scalar Subquery	37
Q15	Monthly Revenue Trend	39
Q16	Referral Sources Delivering >\$100K Net — HAVING	42
Summary	Key Business Findings	44

Introduction

Project 3 is the extraction phase of my DecodeLabs analytics track. Having cleaned the dataset in Project 1 and explored its statistical patterns in Project 2, my goal here was to prove that SQL can interrogate the same 1,200 records with more precision and speed than any spreadsheet pivot table.

SQL is declarative — you describe the output you want and the engine works out how to get it. Every query in this report follows the same logical execution order:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Understanding this sequence is what separates someone who can type SQL from someone who can actually reason about it — and it's the difference that shows up repeatedly across the 16 queries below, especially anywhere WHERE and HAVING could easily be confused.

SQL Concepts Demonstrated Across 16 Queries

Query	Title	Key Concepts
Q1	Overall Business Summary	COUNT, SUM, AVG, ROUND, Aliases
Q2	Orders by Status	GROUP BY, ORDER BY, Percentage Calculation
Q3	Revenue by Referral Source — All Orders	GROUP BY + Multiple Aggregations
Q4	Top Products by Revenue	GROUP BY + Dual AVG Columns
Q5	Referral Source — Successful Orders Only	WHERE + GROUP BY, Boolean Text Filter
Q6	Payment Method Breakdown	GROUP BY on Categorical Column
Q7	Coupon Usage & Realisation Rate	GROUP BY + CASE WHEN Realisation %
Q8	Year-on-Year Realisation Rate	GROUP BY + Realised/Lost Revenue + Rate %
Q9	High Value Orders — Realised vs Unfiltered	WHERE with AND, Comparative Filtering
Q10	High Value Conversion Rate by Channel	CASE WHEN inside Aggregation
Q11	Products with 90+ Successful Orders	WHERE + GROUP BY + HAVING
Q12	Cancel & Return Rate by Product	CASE WHEN Dual Conditional Rates
Q13	Top 10 Customers — Value & Product Mix	CTE (WITH) + JOIN + GROUP BY
Q14	Product Revenue Contribution %	Scalar Subquery + Contribution Percentage
Q15	Monthly Revenue Trend	strftime() Date Grouping
Q16	Referral Sources Delivering >\$100K Net	HAVING on SUM Threshold

Query 2 Orders by Status

Business Question

How are the 1,200 orders split across the five statuses, and what share of the total does each one represent?

SQL

```
SELECT
OrderStatus,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Gross_Value, ROUND((COUNT(*)
* 100.0 / 1200), 2) AS Percentage_of_Total
FROM Orders
GROUP BY OrderStatus
ORDER BY Total_Orders DESC;
```

Query Result

```
1  SELECT
2      OrderStatus,
3      COUNT(*) AS Total_Orders,
4      ROUND(SUM(TotalPrice), 2) AS Gross_Value,
5      ROUND((COUNT(*) * 100.0 / 1200), 2) AS Percentage_of_Total
6  FROM Orders
7  GROUP BY OrderStatus
8  ORDER BY Total_Orders DESC;
```

	OrderStatus	Total_Orders	Gross_Value	Percentage_of_Total
1	Cancelled	250	276396.21	20.83
2	Returned	247	243277.7	20.58
3	Pending	237	256328.15	19.75
4	Shipped	235	246159.58	19.58
5	Delivered	231	242600.32	19.25

Execution finished without errors.

Result Summary

Status	Total Orders	Gross Value	% of Total
Cancelled	250	\$276,396.21	20.83%
Returned	247	\$243,277.70	20.58%
Pending	237	\$256,328.15	19.75%
Shipped	235	\$246,159.58	19.58%
Delivered	231	\$242,600.32	19.25%

Interpretation

The five statuses sit almost evenly between 19% and 21% of total orders, and that's actually the more worrying pattern. A sudden spike is easy to explain away as one bad week. This kind of even spread says the failure is baked into the process itself.

Cancelled is technically the single largest status at 250 orders, ahead of even Delivered at 231. Put Cancelled and Returned together and you get 497 orders, 41.4% of everything, that never become real revenue. No amount of marketing spend fixes that on its own.

Next step: A 10-point reduction in the combined cancel and return rate would unlock roughly \$75,000 in additional realised revenue — the single biggest profitability lever in this dataset.

Query 3 Revenue by Referral Source — All Orders

Business Question

Which marketing channel produces the highest average order value when you look at every order, completed or not?

SQL

```
SELECT
ReferralSource,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Gross_Revenue,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
FROM Orders
GROUP BY ReferralSource
ORDER BY Avg_Order_Value DESC;
```

Query Result

SQL 1*				
1	SELECT			
2	ReferralSource,			
3	COUNT(*) AS Total_Orders,			
4	ROUND(SUM(TotalPrice), 2) AS Gross_Revenue,			
5	ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value			
6	FROM Orders			
7	GROUP BY ReferralSource			
8	ORDER BY Avg_Order_Value DESC;			
	ReferralSource	Total_Orders	Gross_Revenue	Avg_Order_Value
1	Facebook	228	250410.9	1098.29
2	Instagram	259	275285.45	1062.88
3	Email	250	261808.55	1047.23
4	Google	241	250441.48	1039.18
5	Referral	222	226815.58	1021.69
Execution finished without errors.				
Result: 5 rows returned in 25ms				

Result Summary

Referral Source	Total Orders	Gross Revenue	Avg Order Value
Facebook	228	\$250,410.90	\$1,098.29
Instagram	259	\$275,285.45	\$1,062.88
Email	250	\$261,808.55	\$1,047.23
Google	241	\$250,441.48	\$1,039.18
Referral	222	\$226,815.58	\$1,021.69

Interpretation

On the surface, Facebook looks like the standout channel here, leading on average order value at \$1,098.29 despite bringing in the fewest orders of any channel (228). If I stopped at this table, I'd recommend doubling down on Facebook ads.

Instagram leads on volume (259 orders) and total gross revenue (\$275,285.45) but only ranks second on AOV. Query 5 is where this picture changes completely once we filter to orders that actually went through.

Query 4 Top Products by Revenue

Business Question

Which products bring in the most total revenue, and how does the average order value compare to the average unit price across the catalogue?

SQL

```
SELECT
Product,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value,
ROUND(AVG(UnitPrice), 2) AS Avg_Unit_Price
FROM Orders
GROUP BY Product
ORDER BY Total_Revenue DESC;
```

Query Result

SQL 1*

```
1 SELECT
2   Product,
3   COUNT(*) AS Total_Orders,
4   ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
5   ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value,
6   ROUND(AVG(UnitPrice), 2) AS Avg_Unit_Price
7 FROM Orders
8 GROUP BY Product
9 ORDER BY Total_Revenue DESC;
```

	Product	Total_Orders	Total_Revenue	Avg_Order_Value	Avg_Unit_Price
1	Chair	178	195620.11	1098.99	355.66
2	Printer	181	195612.61	1080.73	351.71
3	Laptop	173	192126.56	1110.56	357.71
4	Tablet	179	186568.95	1042.28	367.68
5	Monitor	163	175651.41	1077.62	358.66
6	Desk	170	167459.93	985.06	329.61
7	Phone	156	151722.39	972.58	375.22

Execution finished without errors.
Result: 7 rows returned in 21ms

Result Summary

Product	Total Orders	Total Revenue	Avg Order Value	Avg Unit Price
Chair	178	\$195,620.11	\$1,098.99	\$355.66
Printer	181	\$195,612.61	\$1,080.73	\$351.71
Laptop	173	\$192,126.56	\$1,110.56	\$357.71
Tablet	179	\$186,568.95	\$1,042.28	\$367.68
Monitor	163	\$175,651.41	\$1,077.62	\$358.66
Desk	170	\$167,459.93	\$985.06	\$329.61
Phone	156	\$151,722.39	\$972.58	\$375.22

Interpretation

Chair comes out on top for total revenue (\$195,620.11), combining decent volume with a strong average order value. Printer is almost tied right behind it at \$195,612.61, and actually has the most orders of any product (181).

Phone has the highest average unit price of the whole catalogue (\$375.22) but the lowest total revenue. Customers just don't buy many phones at once per order, which caps the transaction value even though the per-unit cost is the highest. Worth noting the gap between the top product (Chair, \$195K) and the bottom (Phone, \$151K) is only about \$44,000 — nothing here is dragging the catalogue down on its own.

Query 5 Referral Source Performance — Successful Orders Only

Business Question

Once I filter down to the 703 orders that actually generated revenue, which channel delivers the highest realised average order value?

SQL

```
SELECT
ReferralSource,
COUNT(*) AS Successful_Orders,
ROUND(SUM(TotalPrice), 2) AS Net_Revenue,
ROUND(AVG(TotalPrice), 2) AS Realized_AOV
FROM Orders
WHERE IsRealisedRevenue = 'TRUE'
GROUP BY ReferralSource
ORDER BY Realized_AOV DESC;
```

Query Result

SQL 1*

```
1 SELECT
2   ReferralSource,
3   COUNT(*) AS Successful_Orders,
4   ROUND(SUM(TotalPrice), 2) AS Net_Revenue,
5   ROUND(AVG(TotalPrice), 2) AS Realized_AOV
6 FROM Orders
7 WHERE IsRealisedRevenue = 'TRUE'
8 GROUP BY ReferralSource
9 ORDER BY Realized_AOV DESC;
```

	ReferralSource	Successful_Orders	Net_Revenue	Realized_AOV
1	Google	140	155085.42	1107.75
2	Instagram	163	175505.5	1076.72
3	Email	142	147787.01	1040.75
4	Facebook	130	135116.78	1039.36
5	Referral	128	131593.34	1028.07

Execution finished without errors.
Result: 5 rows returned in 13ms

Result Summary

Referral Source	Successful Orders	Net Revenue	Realised AOV
Google	140	\$155,085.42	\$1,107.75
Instagram	163	\$175,505.50	\$1,076.72
Email	142	\$147,787.01	\$1,040.75
Facebook	130	\$135,116.78	\$1,039.36
Referral	128	\$131,593.34	\$1,028.07

Interpretation

This is probably the most important query in the whole project. The moment I add WHERE IsRealisedRevenue = 'TRUE', the entire channel ranking flips. Facebook falls from first place in Query 3 (\$1,098.29 AOV) all the way to fourth here (\$1,039.36) — a drop of \$58.93 per order, the steepest fall of any channel.

Google does the opposite, jumping from fourth to first at \$1,107.75, up \$68.57. Instagram holds second place but actually generates the highest net revenue of any channel (\$175,505.50) by pairing a solid AOV with the most completed orders (163).

This is the WHERE clause execution order playing out in real numbers. Filtering before the GROUP BY and AVG even run changes everything downstream. The unfiltered view in Query 3 was inflating Facebook's AOV with high-value orders that, it turns out, never actually completed.

Worth flagging: Facebook's \$58.93 AOV collapse is the single most actionable finding in this analysis — it points to a targeting or expectations mismatch in how Facebook ads are reaching buyers.

Query 6 Payment Method Breakdown

Business Question

Which payment methods are most popular, and does the method customers choose correlate with how much they spend?

SQL

```
SELECT
PaymentMethod,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
FROM Orders
GROUP BY PaymentMethod
ORDER BY Total_Orders DESC;
```

Query Result

SQL 1*

```
1 SELECT
2     PaymentMethod,
3     COUNT(*) AS Total_Orders,
4     ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
5     ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
6 FROM Orders
7 GROUP BY PaymentMethod
8 ORDER BY Total_Orders DESC;
```

	PaymentMethod	Total_Orders	Total_Revenue	Avg_Order_Value
1	Online	258	262442.94	1017.22
2	Cash	246	259786.29	1056.04
3	Credit Card	234	263847.63	1127.55
4	Debit Card	232	232361.18	1001.56
5	Gift Card	230	246323.92	1070.97

Execution finished without errors.
Result: 5 rows returned in 15ms

Result Summary

Payment Method	Total Orders	Total Revenue	Avg Order Value
Online	258	\$262,442.94	\$1,017.22
Cash	246	\$259,786.29	\$1,056.04
Credit Card	234	\$263,847.63	\$1,127.55
Debit Card	232	\$232,361.18	\$1,001.56
Gift Card	230	\$246,323.92	\$1,070.97

Interpretation

Volume across payment methods is remarkably even — just 28 orders separate the most popular (Online, 258) from the least (Gift Card, 230). No single method dominates customer preference.

Credit Card stands out on value though, leading both AOV (\$1,127.55) and total revenue (\$263,847.63) despite ranking only third on order count. It makes sense — deferred payment tends to loosen people up on big-ticket purchases. Debit Card sits at the other end (\$1,001.56 AOV), which fits a more cautious, pay-as-you-go spending pattern.

Query 7 Coupon Usage & Realisation Rate

Business Question

Which coupons drive the most net revenue, and which one has the highest share of orders that actually go through to completion?

SQL

```
SELECT
Coupon_Clean,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
FROM Orders
GROUP BY Coupon_Clean
ORDER BY Total_Orders DESC;
```

Query Result

SQL 1*

```
1 SELECT
2     Coupon_Clean,
3     COUNT(*) AS Total_Orders,
4     ROUND(SUM(TotalPrice), 2) AS Total_Revenue,
5     ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
6 FROM Orders
7 GROUP BY Coupon_Clean
8 ORDER BY Total_Orders DESC;
```

	Coupon_Clean	Total_Orders	Total_Revenue	Avg_Order_Value
1	FREESHIP	313	335036.99	1070.41
2	NO COUPON	309	322401.41	1043.37
3	WINTER15	292	302483.54	1035.9
4	SAVE10	286	304840.02	1065.87

Execution finished without errors.
Result: 4 rows returned in 18ms

Result Summary

Coupon	Total Orders	Total Revenue	Avg Order Value
FREESHIP	313	\$335,036.99	\$1,070.41
NO COUPON	309	\$322,401.41	\$1,043.37
WINTER15	292	\$302,483.54	\$1,035.90
SAVE10	286	\$304,840.02	\$1,065.87

Interpretation

FREESHIP is used most often (313 orders) and brings in the highest total revenue, but raw volume only tells half the story. What I really want to know is which coupon actually converts, and that needs the realisation rate layered on top, which I calculate below.

Query
7b

Coupon Usage with Realisation Rate (Enhanced)

Business Question

Building on the same coupon breakdown, which one converts orders into actual revenue most reliably?

SQL

```
SELECT
Coupon_Clean AS Coupon,
COUNT(*) AS Total_Orders,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value,
ROUND(SUM(NetRevenue), 2) AS Net_Revenue,
ROUND(100.0 * SUM(CASE WHEN IsRealisedRevenue = 'TRUE'
THEN 1 ELSE 0 END) / COUNT(*), 2) AS Realisation_Pct
FROM Orders
GROUP BY Coupon_Clean
ORDER BY Net_Revenue DESC;
```

Query Result

SQL 1*

```

2   Coupon_Clean AS Coupon,
3   COUNT(*) AS Total_Orders,
4   ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value,
5   ROUND(SUM(NetRevenue), 2) AS Net_Revenue,
6   ROUND(100.0 * SUM(CASE WHEN IsRealisedRevenue = 'TRUE' THEN 1 ELSE 0 END) / COUNT(*), 2) AS Realisation_Pct
7 FROM Orders
8 GROUP BY Coupon_Clean
9 ORDER BY Net_Revenue DESC;
```

	Coupon	Total_Orders	Avg_Order_Value	Net_Revenue	Realisation_Pct
1	SAVE10	286	1065.87	203007.56	63.29
2	FREESHIP	313	1070.41	196107.89	59.11
3	NO COUPON	309	1043.37	185344.29	56.63
4	WINTER15	292	1035.9	160628.31	55.48

Execution finished without errors.
Result: 4 rows returned in 16ms

Result Summary

Coupon	Total Orders	Avg Order Value	Net Revenue	Realisation %
SAVE10	286	\$1,065.87	\$203,007.56	63.29%
FREESHIP	313	\$1,070.41	\$196,107.89	59.11%
NO COUPON	309	\$1,043.37	\$185,344.29	56.63%
WINTER15	292	\$1,035.90	\$160,628.31	55.48%

Interpretation

SAVE10 turns out to be the most efficient coupon once realisation is in the picture — highest conversion rate (63.29%) and highest net revenue (\$203,007.56), even with fewer orders than FREESHIP. FREESHIP drives more volume and a slightly higher AOV but converts at a noticeably lower rate (59.11%).

Even NO COUPON still realises 56.63% of its orders, which tells me coupons in general are only adding about 7 percentage points of conversion on average, not the dramatic lift you'd expect from a discount code. WINTER15 is the weakest performer across every single metric.

Next step: Retire WINTER15, lean further into SAVE10, and test a deeper-discount variant against FREESHIP. The 7-point conversion gap between SAVE10 (63.3%) and NO COUPON (56.6%) is a fair estimate of what the coupon programme is actually worth.

Query 8 Year-on-Year Realisation Rate

Business Question

How has order volume, realised revenue, and the realisation rate moved across the three years in the dataset?

Query (Step 1 — Basic)

```
SELECT
Year,
COUNT(*) AS Total_Orders,
ROUND(SUM(TotalPrice), 2) AS Gross_Revenue,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value,
ROUND(SUM(NetRevenue), 2) AS Net_Revenue
FROM Orders
GROUP BY Year
ORDER BY Year ASC;
```

Query Result

Year	Total_Orders	Gross_Revenue	Avg_Order_Value	Net_Revenue
2023	510	552643.24	1083.61	330547.73
2024	459	480235.87	1046.27	300339.93
2025	231	231882.85	1003.82	114200.39

Execution finished without errors.
Result: 3 rows returned in 14ms

Result Summary

Year	Total Orders	Gross Revenue	Avg Order Value	Net Revenue
2023	510	\$552,643.24	\$1,083.61	\$330,547.73
2024	459	\$480,235.87	\$1,046.27	\$300,339.93
2025	231	\$231,882.85	\$1,003.82	\$114,200.39

Query (Step 2 — Enhanced)

```
SELECT
Year,
COUNT(*) AS Total_Orders,
ROUND(SUM(NetRevenue), 2) AS Realised_Revenue,
ROUND(SUM(LostRevenue), 2) AS Lost_Revenue,
ROUND(SUM(NetRevenue) * 100.0 / (SUM(NetRevenue) +
SUM(LostRevenue)), 2) AS Realisation_Rate_Pct
FROM Orders
GROUP BY Year
ORDER BY Year ASC;
```

Query Result

SQL 1*

```
1 SELECT
2     Year,
3     COUNT(*) AS Total_Orders,
4     ROUND(SUM(NetRevenue), 2) AS Realised_Revenue,
5     ROUND(SUM(LostRevenue), 2) AS Lost_Revenue,
6     ROUND(SUM(NetRevenue) * 100.0 / (SUM(NetRevenue) + SUM(LostRevenue)), 2) AS Realisation_Rate_Pct
7 FROM Orders
8 GROUP BY Year
9 ORDER BY Year ASC;
```

	Year	Total_Orders	Realised_Revenue	Lost_Revenue	Realisation_Rate_Pct
1	2023	510	330547.73	222095.51	59.81
2	2024	459	300339.93	179895.94	62.54
3	2025	231	114200.39	117682.46	49.25

Execution finished without errors.
Result: 3 rows returned in 15ms

Result Summary

Year	Total Orders	Realised Revenue	Lost Revenue	Realisation Rate %
2023	510	\$330,547.73	\$222,095.51	59.81%
2024	459	\$300,339.93	\$179,895.94	62.54%
2025	231	\$114,200.39	\$117,682.46	49.25%

Interpretation

I started with the basic year-on-year view (above) before adding lost revenue and an explicit realisation rate to get the full picture. That second pass is where the real story shows up: the realisation rate fell off a cliff, from 62.54% in 2024 to 49.25% in 2025. Roughly every second dollar of 2025 demand is now being lost before it ever reaches the customer.

Order volume was already trending down too, from 510 in 2023 to 459 in 2024, even while the realisation rate was still improving slightly. That makes the 2025 collapse look like a sudden operational shift rather than a slow drift. Something changed after Q4 2024.

Both growth levers, volume and conversion, are now moving in the wrong direction at the same time, which is what makes this the most urgent finding in the report.

Worth flagging: 2025 only covers January through June, so the trend has time to be addressed. Audit any process changes introduced after Q4 2024 — payment gateway, courier partner, return policy — to find what triggered the collapse.

Query
9a

High Value Orders — Realised Only (>\$3,000)

Business Question

Of all orders above \$3,000, which ones actually completed, and what patterns show up across product, channel, and payment method?

SQL

```
SELECT
OrderID,
Product,
ReferralSource,
PaymentMethod,
OrderStatus,
TotalPrice
FROM Orders
WHERE TotalPrice > 3000
AND IsRealisedRevenue = 'TRUE'
ORDER BY TotalPrice DESC;
```

Query Result

SQL 1*

3	Product,
4	ReferralSource,
5	PaymentMethod,
6	OrderStatus,

	OrderID	Product	ReferralSource	PaymentMethod	OrderStatus	TotalPrice
1	ORD200789	Tablet	Email	Online	Delivered	3456.4
2	ORD200632	Laptop	Facebook	Gift Card	Delivered	3390.8
3	ORD200107	Printer	Instagram	Gift Card	Shipped	3353.75
4	ORD201065	Printer	Referral	Debit Card	Delivered	3334
5	ORD201031	Phone	Email	Debit Card	Pending	3322.55
6	ORD200463	Laptop	Instagram	Debit Card	Shipped	3313.9
7	ORD200361	Printer	Facebook	Credit Card	Delivered	3299.25
8	ORD200367	Laptop	Instagram	Gift Card	Pending	3293.85
9	ORD200837	Chair	Email	Online	Delivered	3277.75
10	ORD200540	Laptop	Referral	Online	Pending	3243.25
11	ORD200221	Tablet	Google	Debit Card	Pending	3196.85
12	ORD201079	Phone	Email	Gift Card	Delivered	3170
13	ORD200097	Tablet	Email	Debit Card	Pending	3097.6
14	ORD200241	Chair	Email	Online	Pending	3078.35
15	ORD200876	Chair	Google	Gift Card	Pending	3044.55
16	ORD200492	Laptop	Instagram	Debit Card	Shipped	3032.6

Result Summary

OrderID	Product	Referral	Payment	Status	Total
ORD200789	Tablet	Email	Online	Delivered	3456.40
ORD200632	Laptop	Facebook	Gift Card	Delivered	3390.80
ORD200107	Printer	Instagram	Gift Card	Shipped	3353.75

OrderID	Product	Referral	Payment	Status	Total
ORD201065	Printer	Referral	Debit Card	Delivered	3334.00
ORD201031	Phone	Email	Debit Card	Pending	3322.55
ORD200463	Laptop	Instagram	Debit Card	Shipped	3313.90
ORD200361	Printer	Facebook	Credit Card	Delivered	3299.25
ORD200367	Laptop	Instagram	Gift Card	Pending	3293.85
ORD200837	Chair	Email	Online	Delivered	3277.75
ORD200540	Laptop	Referral	Online	Pending	3243.25
ORD200221	Tablet	Google	Debit Card	Pending	3196.85
ORD201079	Phone	Email	Gift Card	Delivered	3170.00
ORD200097	Tablet	Email	Debit Card	Pending	3097.60
ORD200241	Chair	Email	Online	Pending	3078.35
ORD200876	Chair	Google	Gift Card	Pending	3044.55
ORD200492	Laptop	Instagram	Debit Card	Shipped	3032.60

Query
9b

High Value Orders — Full Unfiltered List (>\$3,000)

Business Question

The same \$3,000+ threshold, but without the realised-only filter, so I can see everything that fell through.

SQL

```
SELECT
OrderID,
Product,
ReferralSource,
PaymentMethod,
OrderStatus,
TotalPrice
FROM Orders
WHERE TotalPrice > 3000
ORDER BY TotalPrice DESC;
```

Query Result

SQL 1*

```

7      TotalPrice
8  FROM Orders
9  WHERE TotalPrice > 3000
10 ORDER BY TotalPrice DESC;
```

	OrderID	Product	ReferralSource	PaymentMethod	OrderStatus	TotalPrice
1	ORD200789	Tablet	Email	Online	Delivered	3456.4
2	ORD201122	Monitor	Facebook	Online	Returned	3390.95
3	ORD200632	Laptop	Facebook	Gift Card	Delivered	3390.8
4	ORD200469	Chair	Facebook	Cash	Cancelled	3384.9
5	ORD200328	Tablet	Google	Online	Cancelled	3370.2
6	ORD200107	Printer	Instagram	Gift Card	Shipped	3353.75
7	ORD200326	Laptop	Facebook	Gift Card	Returned	3352.4
8	ORD201065	Printer	Referral	Debit Card	Delivered	3334
9	ORD201031	Phone	Email	Debit Card	Pending	3322.55
10	ORD200463	Laptop	Instagram	Debit Card	Shipped	3313.9
11	ORD200361	Printer	Facebook	Credit Card	Delivered	3299.25
12	ORD200367	Laptop	Instagram	Gift Card	Pending	3293.85
13	ORD200837	Chair	Email	Online	Delivered	3277.75
14	ORD200527	Chair	Referral	Credit Card	Cancelled	3267.35
15	ORD200768	Tablet	Email	Cash	Cancelled	3267.3
16	ORD200889	Monitor	Email	Credit Card	Cancelled	3253.6
17	ORD200540	Laptop	Referral	Online	Pending	3243.25

Result Summary

OrderID	Product	Referral	Payment	Status	Total
ORD200789	Tablet	Email	Online	Delivered	3456.40
ORD201122	Monitor	Facebook	Online	Returned	3390.95
ORD200632	Laptop	Facebook	Gift Card	Delivered	3390.80
ORD200469	Chair	Facebook	Cash	Cancelled	3384.90

OrderID	Product	Referral	Payment	Status	Total
ORD200328	Tablet	Google	Online	Cancelled	3370.20
ORD200107	Printer	Instagram	Gift Card	Shipped	3353.75
ORD200326	Laptop	Facebook	Gift Card	Returned	3352.40
ORD201065	Printer	Referral	Debit Card	Delivered	3334.00
ORD201031	Phone	Email	Debit Card	Pending	3322.55
ORD200463	Laptop	Instagram	Debit Card	Shipped	3313.90
ORD200361	Printer	Facebook	Credit Card	Delivered	3299.25
ORD200367	Laptop	Instagram	Gift Card	Pending	3293.85
ORD200837	Chair	Email	Online	Delivered	3277.75
ORD200527	Chair	Referral	Credit Card	Cancelled	3267.35
ORD200768	Tablet	Email	Cash	Cancelled	3267.30
ORD200889	Monitor	Email	Credit Card	Cancelled	3253.60
ORD200540	Laptop	Referral	Online	Pending	3243.25
ORD200802	Chair	Email	Gift Card	Cancelled	3223.20
ORD200957	Monitor	Facebook	Cash	Returned	3219.45
ORD200086	Printer	Referral	Online	Cancelled	3215.15
ORD200221	Tablet	Google	Debit Card	Pending	3196.85
ORD200967	Tablet	Google	Credit Card	Returned	3194.60
ORD200296	Desk	Referral	Debit Card	Cancelled	3194.00
ORD201079	Phone	Email	Gift Card	Delivered	3170.00
ORD200364	Phone	Facebook	Cash	Cancelled	3143.70
ORD200764	Laptop	Email	Credit Card	Cancelled	3137.15
ORD200010	Tablet	Instagram	Credit Card	Returned	3129.85
ORD200820	Phone	Email	Cash	Returned	3119.05
ORD200097	Tablet	Email	Debit Card	Pending	3097.60
ORD200241	Chair	Email	Online	Pending	3078.35
ORD200450	Monitor	Instagram	Gift Card	Returned	3075.50
ORD200876	Chair	Google	Gift Card	Pending	3044.55

OrderID	Product	Referral	Payment	Status	Total
ORD200492	Laptop	Instagram	Debit Card	Shipped	3032.60
ORD200633	Laptop	Instagram	Gift Card	Cancelled	3008.60

Interpretation

Putting the two queries side by side is the point here. Out of 34 orders above \$3,000, only 16 made it through to realised revenue, a 47.1% conversion rate, which is worse than the overall dataset average of roughly 58.6%. The biggest orders in the entire dataset are failing more often than typical ones, not less.

The AND operator in 9a is doing real work here: `TotalPrice > 3000 AND IsRealisedRevenue = 'TRUE'` has to satisfy both conditions at once, which is compound filtering happening at execution Step 2, before any grouping occurs.

Email and Instagram show up repeatedly in the realised list (9a), reinforcing what I saw in Query 5 — these two channels are the most reliable for bringing in serious, committed buyers rather than just window shoppers.

Query
10

High Value Conversion Rate by Channel

Business Question

For orders above \$3,000, what percentage does each referral channel actually convert, and which one is losing the most premium business?

SQL

```
SELECT
ReferralSource,
COUNT(*) AS High_Value_Orders,
SUM(CASE WHEN IsRealisedRevenue = 'TRUE' THEN 1 ELSE 0 END) AS Realised,
SUM(CASE WHEN IsRealisedRevenue = 'FALSE' THEN 1 ELSE 0 END) AS Lost,
ROUND(SUM(CASE WHEN IsRealisedRevenue = 'TRUE' THEN 1 ELSE 0
END) * 100.0 / COUNT(*), 2) AS Conversion_Rate_Pct
FROM Orders
WHERE TotalPrice > 3000
GROUP BY ReferralSource
ORDER BY Conversion_Rate_Pct DESC;
```

Query Result

ReferralSource	Top_High_Value_Orders	Realised	Lost	Conversion_Rate_Pct
1 Instagram	7	4	3	57.14
2 Email	11	6	5	54.55
3 Google	4	2	2	50.0
4 Referral	5	2	3	40.0
5 Facebook	7	2	5	28.57

Execution finished without errors.
Result: 5 rows returned in 15ms

Result Summary

Referral Source	High Value Orders	Realised	Lost	Conversion %
Instagram	7	4	3	57.14%
Email	11	6	5	54.55%
Google	4	2	2	50.00%
Referral	5	2	3	40.00%
Facebook	7	2	5	28.57%

Interpretation

Facebook loses five out of every seven high-value orders it generates, a 28.57% conversion rate on its most premium transactions. This is the clearest proof yet of the Facebook targeting issue that first showed up back in Query 3 and Query 5.

Email brings in the most high-value orders overall (11) and converts a little over half (54.55%). Instagram has the best conversion rate of any channel here (57.14%), confirming it as the most dependable source of serious, premium buyers.

Wrapping CASE WHEN inside an aggregate is what makes this possible in a single query — rather than running three separate queries to get the realised count, the lost count, and the rate, I can calculate all of it inline within one GROUP BY pass.

Worth flagging: Facebook and Instagram generate the same number of high-value orders (7 each) but Facebook converts at less than half Instagram's rate. This isn't a volume problem, it's a targeting and post-checkout experience problem.

Query
11

Successful Products with 90+ Completed Orders — HAVING

Business Question

Among successful orders only, which products have crossed 90 completions, and how do their realised average order values compare?

SQL

```
SELECT
Product,
COUNT(*) AS Total_Orders,
ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
FROM Orders WHERE
IsRealisedRevenue = 'TRUE'
GROUP BY Product
HAVING COUNT(*) > 90
ORDER BY Avg_Order_Value DESC;
```

Query Result

SQL 1*

```
1  SELECT
2      Product,
3      COUNT(*) AS Total_Orders,
4      ROUND(AVG(TotalPrice), 2) AS Avg_Order_Value
5  FROM Orders
6  WHERE IsRealisedRevenue = 'TRUE'
7  GROUP BY Product
8  HAVING COUNT(*) > 90
9  ORDER BY Avg_Order_Value DESC;
```

	Product	Total_Orders	Avg_Order_Value
1	Chair	105	1145.91
2	Printer	108	1120.36
3	Laptop	99	1098.09
4	Monitor	92	1065.97
5	Tablet	102	1013.85
6	Phone	94	1005.69
7	Desk	103	961.56

Execution finished without errors.
Result: 7 rows returned in 12ms

Result Summary

Product	Total Orders	Avg Order Value
Chair	105	\$1,145.91
Printer	108	\$1,120.36
Laptop	99	\$1,098.09
Monitor	92	\$1,065.97
Tablet	102	\$1,013.85
Phone	94	\$1,005.69
Desk	103	\$961.56

Interpretation

All seven products clear the 90-order threshold on completed orders alone, which is a genuinely good sign — every product has a solid base of customers who actually follow through. Chair leads on realised AOV (\$1,145.91), and Printer has both the highest order count (108) and the second-best AOV (\$1,120.36). Desk is the only product averaging under \$1,000 on completed transactions (\$961.56).

HAVING is doing something WHERE simply cannot do here. COUNT(*) only exists once GROUP BY has formed the buckets at execution Step 3, but WHERE runs earlier at Step 2, before any grouping has happened. HAVING runs after the buckets exist, at Step 4, which makes it the only clause that can filter on an aggregated value like COUNT(*).

**Query
12****Cancellation & Return Rate by Product****Business Question**

Which products have the worst cancellation rates (a pre-shipment problem) versus the worst return rates (a post-delivery problem)?

SQL

```
SELECT
Product,
COUNT(*) AS Total_Orders,
ROUND(100.0 * SUM(CASE WHEN IsCancelled = 'TRUE'
THEN 1 ELSE 0 END) / COUNT(*), 2) AS Cancel_Rate_Pct,
ROUND(100.0 * SUM(CASE WHEN IsReturned = 'TRUE' THEN 1
ELSE 0 END) / COUNT(*), 2) AS Return_Rate_Pct
FROM Orders
GROUP BY Product
ORDER BY Cancel_Rate_Pct DESC;
```

Query Result

```
SQL 1*
1  SELECT
2      Product,
3      COUNT(*) AS Total_Orders,
4      ROUND(100.0 * SUM(CASE WHEN IsCancelled = 'TRUE' THEN 1 ELSE 0 END) / COUNT(*), 2) AS Cancel_Rate_Pct,
5      ROUND(100.0 * SUM(CASE WHEN IsReturned = 'TRUE' THEN 1 ELSE 0 END) / COUNT(*), 2) AS Return_Rate_Pct
6  FROM Orders
7  GROUP BY Product
8  ORDER BY Cancel_Rate_Pct DESC;
```

	Product	Total_Orders	Cancel_Rate_Pct	Return_Rate_Pct
1	Chair	178	25.28	15.73
2	Monitor	163	21.47	22.09
3	Desk	170	20.59	18.82
4	Laptop	173	20.23	22.54
5	Phone	156	19.87	19.87
6	Printer	181	19.34	20.99
7	Tablet	179	18.99	24.02

Execution finished without errors.
Result: 7 rows returned in 90ms

Result Summary

Product	Total Orders	Cancel Rate %	Return Rate %
Chair	178	25.28%	15.73%
Monitor	163	21.47%	22.09%
Desk	170	20.59%	18.82%
Laptop	173	20.23%	22.54%
Phone	156	19.87%	19.87%
Printer	181	19.34%	20.99%
Tablet	179	18.99%	24.02%

Interpretation

Chair has the worst cancel rate in the catalogue (25.28%), which is a pre-shipment problem. Customers are placing the order and then backing out before it ever ships, which usually points to price shock at checkout, stock-out disappointment, or buyer's remorse during a long fulfilment window.

Tablet has the worst return rate instead (24.02%), which is a post-delivery problem — customers receive it and send it back, suggesting a gap between what the listing promises and what actually arrives.

These are two genuinely different failure modes and they need two different fixes. Chair needs less checkout friction and clearer payment confirmation. Tablet needs better product photography, more accurate descriptions, and a better unboxing experience.

Worth flagging: Chair and Tablet need attention first, but for completely different reasons. Treating them with the same fix would solve neither one.

**Query
13a****Top 10 Customers by Net Revenue****Business Question**

Who are the highest-value customers in the dataset, and how many orders has each actually placed?

SQL

```
SELECT
CustomerID,
COUNT(*) AS Total_Orders,
ROUND(SUM(NetRevenue), 2) AS Net_Revenue
FROM Orders
GROUP BY CustomerID
ORDER BY Net_Revenue
DESC
LIMIT 10;
```

Query Result

SQL 1*

1

2

3

4

5

6

7

8

```

SELECT
    CustomerID,
    COUNT(*) AS Total_Orders,
    ROUND(SUM(NetRevenue), 2) AS Net_Revenue
FROM Orders
GROUP BY CustomerID
ORDER BY Net_Revenue DESC
LIMIT 10;

```

	CustomerID	Total_Orders	Net_Revenue
1	C57276	1	3456.4
2	C67260	1	3390.8
3	C16775	1	3353.75
4	C47778	1	3334.0
5	C59183	1	3322.55
6	C25276	1	3313.9
7	C53464	1	3299.25
8	C13108	1	3293.85
9	C88029	1	3277.75
10	C87281	1	3243.25

Execution finished without errors.

Result: 10 rows returned in 21ms

Result Summary

CustomerID	Total Orders	Net Revenue
C57276	1	\$3,456.40
C67260	1	\$3,390.80
C16775	1	\$3,353.75
C47778	1	\$3,334.00
C59183	1	\$3,322.55
C53464	1	\$3,299.25

CustomerID	Total Orders	Net Revenue
C25276	1	\$3,313.90
C13108	1	\$3,293.85
C88029	1	\$3,277.75
C87281	1	\$3,243.25

Interpretation

Every single customer in the top 10 placed exactly one order, all clustered between roughly \$3,200 and \$3,500. There's zero repeat business in the premium segment. The business is clearly capable of acquiring high-spending customers, it just isn't retaining a single one of them.

This is probably the most overlooked finding in the whole dataset. The entire top-end revenue base is built on first-time buyers, with no loyalty mechanism, no repeat-purchase flow, and no real customer lifetime value being captured beyond that first transaction.

**Query
13b****What Are the Top 10 Customers Actually Buying? — CTE + JOIN****Business Question**

Following up on Q13a: now that I know who the top 10 customers are, what product mix are they buying, and is there a pattern worth acting on?

SQL

```
WITH Top_10_Customers AS (  
  SELECT CustomerID  
  FROM Orders  
  GROUP BY CustomerID ORDER BY  
  SUM(NetRevenue) DESC  
  LIMIT 10  
)  
SELECT  
  o.CustomerID,  
  o.Product, COUNT(*) AS Purchase_Count,  
  ROUND(SUM(o.NetRevenue), 2) AS  
  Product_Net_Revenue  
FROM Orders o JOIN Top_10_Customers tc ON o.CustomerID  
= tc.CustomerID  
GROUP BY o.CustomerID, o.Product  
ORDER BY o.CustomerID ASC, Purchase_Count DESC;
```

Query Result

SQL 1*

```
1 WITH Top_10_Customers AS (  
2   SELECT CustomerID  
3   FROM Orders  
4   GROUP BY CustomerID  
5   ORDER BY SUM(NetRevenue) DESC  
6   LIMIT 10  
7 )  
8 SELECT  
9   o.CustomerID,  
10  o.Product,  
11  o.Purchase_Count,  
12  ROUND(SUM(o.NetRevenue), 2) AS Product_Net_Revenue  
13 FROM Orders o JOIN Top_10_Customers tc ON o.CustomerID  
14 = tc.CustomerID  
15 GROUP BY o.CustomerID, o.Product  
16 ORDER BY o.CustomerID ASC, Purchase_Count DESC;
```

	CustomerID	Product	Purchase_Count	Product_Net_Revenue
1	C13108	Laptop	1	3293.85
2	C16775	Printer	1	3353.75
3	C25276	Laptop	1	3313.9
4	C47778	Printer	1	3334.0
5	C53464	Printer	1	3299.25
6	C57276	Tablet	1	3456.4
7	C59183	Phone	1	3322.55
8	C67260	Laptop	1	3390.8
9	C87281	Laptop	1	3243.25
10	C88029	Chair	1	3277.75

Result Summary

CustomerID	Product	Purchase Count	Product Net Revenue
C13108	Laptop	1	3293.85
C16775	Printer	1	3353.75
C25276	Laptop	1	3313.90
C47778	Printer	1	3334.00
C53464	Printer	1	3299.25
C57276	Tablet	1	3456.40
C59183	Phone	1	3322.55
C67260	Laptop	1	3390.80
C87281	Laptop	1	3243.25
C88029	Chair	1	3277.75

Interpretation

I wanted to go one layer deeper than Q13a. Knowing who the top customers are is useful, but knowing what they're buying is what actually opens up a next step. So, I wrote a CTE (WITH) to first isolate the top 10 customer IDs, then joined that back against the full Orders table to break down each customer's purchases by product.

Laptops and Printers dominate this group — 4 laptop purchases and 3 printer purchases between them, out of just 10 customers. That's not a coincidence. These are durable, infrequent-replacement products. Nobody buys a new laptop or printer every few months, so it's reasonable to expect these customers won't need the same product again for at least a year or two.

What that means in practice: good after-sale service and a deliberate follow-up sequence could realistically bring these same high-value customers back, not for another laptop, but for accessories, complementary products, or whatever else fits naturally around what they've already bought. The opportunity isn't to re-sell the same item, it's to widen the basket on the back of a relationship that already exists.

On the SQL side, the CTE keeps the logic readable: `Top_10_Customers` is computed once and named, then the JOIN pulls in every order from those specific customers without repeating the ranking logic inline. It's a cleaner way to chain a filtered subset into a second-level query than nesting subqueries directly.

Next step: Build a post-purchase nurture sequence for these 10 customers specifically — thank-you follow-up, accessory bundle offer, or a check-in roughly a year after the laptop or printer purchase. The product mix tells me exactly when and what to offer.

Query
14

Product Revenue Contribution % — Scalar Subquery

Business Question

What percentage of total net revenue does each product contribute, and how concentrated or spread out is the catalogue?

SQL

```
SELECT
Product, ROUND(SUM(NetRevenue), 2) AS
Net_Revenue,
ROUND(100.0 * SUM(NetRevenue) / (SELECT SUM(NetRevenue)
FROM Orders), 2) AS Contribution_Pct
FROM Orders
GROUP BY Product
ORDER BY Contribution_Pct DESC;
```

Query Result

SQL 1*

```
1  SELECT
2      Product,
3      ROUND(SUM(NetRevenue), 2) AS Net_Revenue,
4      ROUND(100.0 * SUM(NetRevenue) / (SELECT SUM(NetRevenue) FROM Orders), 2) AS Contribution_Pct
5  FROM Orders
6  GROUP BY Product
7  ORDER BY Contribution_Pct DESC;
```

	Product	Net_Revenue	Contribution_Pct
1	Printer	120998.36	16.24
2	Chair	120320.97	16.15
3	Laptop	108710.54	14.59
4	Tablet	103413.13	13.88
5	Desk	99040.75	13.29
6	Monitor	98069.37	13.16
7	Phone	94534.93	12.69

Execution finished without errors.
Result: 7 rows returned in 13ms

Result Summary

Product	Net Revenue	Contribution %
Printer	\$120,998.36	16.24%
Chair	\$120,320.97	16.15%
Laptop	\$108,710.54	14.59%
Tablet	\$103,413.13	13.88%
Desk	\$99,040.75	13.29%
Monitor	\$98,069.37	13.16%
Phone	\$94,534.93	12.69%

Interpretation

No single product clears 16.24% of total net revenue, and the spread from top to bottom is only about 3.5 percentage points (Printer at 16.24% vs Phone at 12.69%). This is a genuinely well-diversified catalogue with no real concentration risk sitting on any one SKU.

The scalar subquery here, `SELECT SUM(NetRevenue) FROM Orders`, calculates the grand total once and reuses it as the denominator for every product row. That's a subquery embedded directly inside a `SELECT` expression rather than inside `WHERE`.

Diversification is good for risk, but it also means there's no obvious hero product to rally around. Printer and Chair, sitting almost neck-and-neck at the top, are the natural candidates if I wanted to invest in bundles, accessories, or service plans to push one of them into a clearer leadership position.

**Query
15****Monthly Revenue Trend****Business Question**

How does net revenue move month to month across all three years, and are the 2025 months tracking above or below where they landed in 2024?

SQL

```
SELECT
strftime('%Y-%m', Date) AS Month, COUNT(*) AS
Total_Orders, ROUND(SUM(NetRevenue), 2) AS Net_Revenue
FROM Orders
GROUP BY Month
ORDER BY Month ASC;
```

Query Result

SQL 1*

1	SELECT
2	strftime('%Y-%m', Date) AS Month,
3	COUNT(*) AS Total_Orders,
4	ROUND(SUM(NetRevenue), 2) AS Net_Revenue

	Month	Total_Orders	Net_Revenue
1	2023-01	47	33987.41
2	2023-02	37	17460.67
3	2023-03	43	24658.77
4	2023-04	31	18846.48
5	2023-05	49	40347.7
6	2023-06	45	24629.09
7	2023-07	44	34241.91
8	2023-08	51	29552.46
9	2023-09	29	13939.85
10	2023-10	47	40323.75
11	2023-11	41	26641.56
12	2023-12	46	25918.08
13	2024-01	32	28234.16
14	2024-02	32	22402.83
15	2024-03	36	22334.89
16	2024-04	50	29904.48
17	2024-05	34	16937.69

Result Summary

Month	Total Orders	Net Revenue
2023-01	47	33987.41
2023-02	37	17460.67
2023-03	43	24658.77
2023-04	31	18846.48
2023-05	49	40347.70
2023-06	45	24629.09

Month	Total Orders	Net Revenue
2023-07	44	34241.91
2023-08	51	29552.46
2023-09	29	13939.85
2023-10	47	40323.75
2023-11	41	26641.56
2023-12	46	25918.08
2024-01	32	28234.16
2024-02	32	22402.83
2024-03	36	22334.89
2024-04	50	29904.48
2024-05	34	16937.69
2024-06	53	37647.69
2024-07	43	24731.41
2024-08	28	11077.38
2024-09	44	29899.21
2024-10	31	26566.40
2024-11	35	20471.43
2024-12	41	30132.36
2025-01	27	18424.28
2025-02	37	14167.80
2025-03	49	19502.47
2025-04	32	7708.18
2025-05	37	20669.68
2025-06	49	33727.98

Interpretation

This query returns 30 rows, January 2023 through June 2025, and monthly revenue swings hard, from \$7,708.18 in April 2025 up to \$40,347.70 in May 2023. There's no clean seasonal pattern holding consistently across all three years.

Most 2025 months are sitting noticeably below their 2024 equivalents, which lines up exactly with the realisation rate collapse I flagged in Query 8. April 2025 stands out as the clearest outlier, the single worst month in the entire dataset by a wide margin.

`strftime('%Y-%m', Date)` is what groups every row by year and month together, doing the job that `MONTH()` and `YEAR()` functions handle in MySQL or PostgreSQL. It's the most granular time grouping the Date column supports.

Next step: April 2025 deserves a closer, standalone look since it may explain a large part of the wider 2025 collapse on its own. A rolling 3-month forecast with a minimum-revenue alert would catch the next anomaly like this much earlier.

**Query
16****Referral Sources Delivering >\$100K Net Revenue — HAVING****Business Question**

Which referral channels clear the \$100,000 net revenue mark, and how does HAVING differ from WHERE when filtering this kind of result?

SQL

```
SELECT
ReferralSource,
COUNT(*) AS Orders, ROUND(SUM(NetRevenue), 2) AS
Net_Revenue
FROM Orders
GROUP BY ReferralSource
HAVING SUM(NetRevenue) > 100000
ORDER BY Net_Revenue DESC;
```

Query Result

SQL 1*

```
1 SELECT
2 ReferralSource,
3 COUNT(*) AS Orders, ROUND(SUM(NetRevenue), 2) AS Net_Revenue
4 FROM Orders
5 GROUP BY ReferralSource
6 HAVING SUM(NetRevenue) > 100000
7 ORDER BY Net_Revenue DESC;
8
```

	ReferralSource	Orders	Net_Revenue
1	Instagram	259	175505.5
2	Google	241	155085.42
3	Email	250	147787.01
4	Facebook	228	135116.78
5	Referral	222	131593.34

Execution finished without errors.
Result: 5 rows returned in 24ms

Result Summary

Referral Source	Orders	Net Revenue
Instagram	259	\$175,505.50
Google	241	\$155,085.42
Email	250	\$147,787.01
Facebook	228	\$135,116.78
Referral	222	\$131,593.34

Interpretation

All five referral sources clear the \$100K net revenue threshold, which is a genuinely strong result, no channel is underperforming in absolute terms. Instagram leads at \$175,505.50, followed by Google at \$155,085.42. Together those two alone account for around 44% of total net revenue.

HAVING SUM(NetRevenue) > 100000 cannot be rewritten as a WHERE clause. SUM(NetRevenue) only exists once GROUP BY has formed the buckets at Step 3, and WHERE runs earlier at Step 2, before any aggregate value exists to filter on. HAVING is the only clause that can filter on the output of an aggregate function like SUM.

Next step: Shift incremental marketing budget toward Instagram and Google. Protect Email as the most cost-efficient channel above \$100K since it's an owned audience with no ad spend attached. Improve Facebook and Referral targeting before scaling spend on either.

Summary of Key Findings

Across 16 SQL queries, here are the findings I'd actually act on:

1. The 41.4% Revenue Leakage

497 of 1,200 orders were cancelled or returned. Cancelled (250) is the single most common order status, ahead of even Delivered (231). A 10-point reduction in the combined cancel and return rate would unlock roughly \$75,000 in additional realised revenue.

2. The Facebook Targeting Paradox

Facebook leads every channel on unfiltered AOV (\$1,098.29) but drops to fourth (\$1,039.36) once filtered to realised orders. On orders above \$3,000 specifically, Facebook converts at only 28.57% versus Instagram's 57.14%, losing 5 of every 7 premium orders it brings in.

3. The 2025 Realisation Collapse

The realisation rate fell from 62.54% in 2024 to 49.25% in 2025, every second dollar of demand now being lost. April 2025 produced only \$7,708 in net revenue, the worst single month in the dataset, and it likely accounts for a large share of the wider 2025 decline.

4. Zero Repeat Business at the Top

Every one of the top 10 customers placed exactly one order. Digging into what they bought (Q13b) shows 4 laptops and 3 printers among them, durable products people don't repurchase for years, which makes a follow-up nurture sequence a clear, specific opportunity rather than a vague loyalty idea.

5. Chair Cancels, Tablet Returns

Chair has the worst cancel rate (25.28%), a pre-shipment problem. Tablet has the worst return rate (24.02%), a post-delivery problem. Two different failure modes that need two different fixes.

6. SAVE10 Beats FREESHIP on Conversion

FREESHIP wins on volume and AOV, but SAVE10 delivers the highest realisation rate (63.29%) and highest net revenue (\$203,007.56). WINTER15 is the weakest performer on every metric and is the clearest candidate to retire.

7. A Well-Diversified Catalogue, No Hero SKU

No product exceeds 16.24% of net revenue, and the spread top to bottom is just 3.55 points. Printer and Chair are the natural candidates to invest in if the goal is to build a clearer leading product.

8. Credit Card Buyers Spend 12.6% More

Credit Card customers average \$1,127.55 per order versus \$1,001.56 for Debit Card, a \$126 gap. Nudging checkout toward Credit Card or Gift Card is a low-cost way to lift AOV without touching the product mix at all.

"Project 3 is not about viewing spreadsheets — it is about Querying for Truth."

— DecodeLabs